

The Clerk's Arithmetic Manual

*How to do math on numbers too big to hold in your head -
by picturing machines you can hold in your head.*

The machine: a four-wheel odometer

Forget bits for a moment. Picture the thing in your car that counts miles: a row of wheels, each wheel marked 0 through 9, and when a wheel rolls past 9 it snaps back to 0 and clicks the wheel to its left forward one notch. That's mechanical addition - nobody thinks, the wheels do it.

Our clerk works with the same machine, except:

- each wheel has **2^{64} positions** instead of 10 (about 1.8×10^{19} - a wheel with more markings than there are grains of sand on Earth), and
- there are exactly **four wheels**.

wheel 3	wheel 2	wheel 1	wheel 0
+-----+	+-----+	+-----+	+-----+
042	917	551	333
+-----+	+-----+	+-----+	+-----+
big end			small end

the number = $w_3 \cdot B^3 + w_2 \cdot B^2 + w_1 \cdot B + w_0$ where $B = 2^{64}$

Each wheel is one `uint64_t` - one limb. Wheel 0 is the "ones" wheel.

The one rule of the room: there is no fifth wheel. Whatever would have gone on wheel 4 just doesn't happen - the odometer rolls back to zero and keeps going. Biggest possible reading plus one tick = all zeros. Mathematicians call this $\text{mod } 2^{256}$; a mechanic calls it *rolling the odometer*. Every operation below is a consequence of this one picture.

Adding: turn the wheels

To add two numbers you add the wheels pairwise, right to left, and every time a wheel rolls past its top it clicks the next wheel forward.

```
wheel 0:  a0 + b0
           |
           +-- did it roll past max? -> click a 1 into wheel 1
```

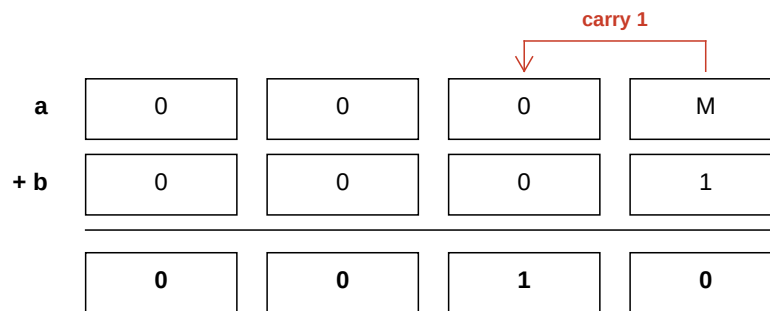
How do you *detect* the click? When a wheel wraps, its new position is **lower than what you added** - you added 5 and now it reads 2? It must have gone all the way around. In code: `sum < a` means "it wrapped."

Walkthrough - `U(M, 0, 0, 0) + 1` (wheel 0 maxed out, $M = 2^{64} - 1$):

```
wheel 0:  M + 1 -> 0, CLICK      (it read M, we added 1, now 0)
```

```
wheel 1: 0 + 0 + click -> 1
wheels 2,3: unchanged

result: U(0, 1, 0, 0) = 2^64 [checkmark]
```



The click out of wheel 3 has nowhere to go - the odometer rolls over.
That's the whole implementation: 4 adds, 3 carries, one forgiven drop.

Subtracting: wind backwards

Subtraction is the same machine wound the other way. When a wheel needs to go below 0 it can't - so it borrows a full turn from the wheel to its left: the left wheel winds back one notch, the right wheel wraps around to nearly-max and continues.

Walkthrough - `U(0, 1, 0, 0) - 1`` (i.e. $2^{64} - 1$):

```
wheel 0: 0 - 1 -> must borrow: reads M, left wheel loses 1
wheel 1: 1 - 1 (the debt) -> 0

result: U(M, 0, 0, 0) = 2^64 - 1 [checkmark]
```

And the beautiful part - **the last borrow is forgiven.** `0 - 1`` rolls the whole odometer backwards one tick and it reads all-max: `0xFFFF...``. The machine can't represent "negative one"; it can only show the position one tick before zero. Which is exactly -1's identity.

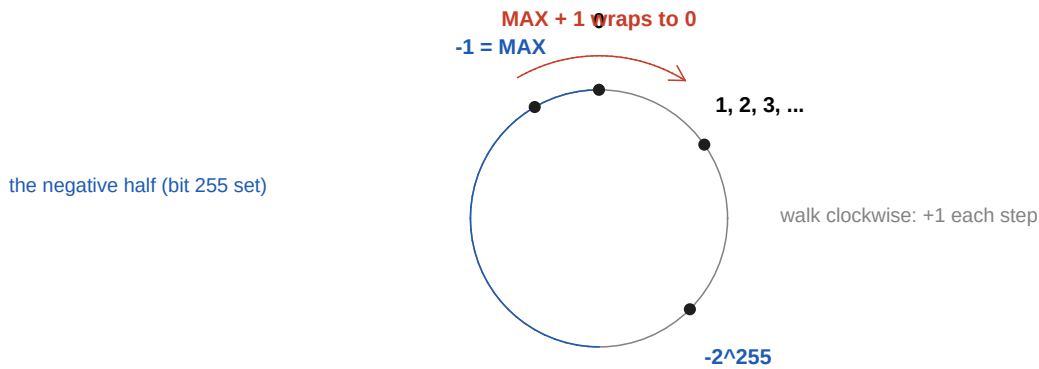
The clock face: where negatives live

This deserves its own picture because it's the deepest idea in the manual. A four-wheel odometer isn't a number line - it's a **clock**:

```

      0
      |
-2^255  ----+----  +2^255-1
(0x80..00)  |      (0x7F..FF)
      |
    the rest of the dial:
    MAX, MAX-1, ... = -1, -2, ...

```



Every reading is simultaneously two numbers: what it says unsigned, and what it says if you read "one tick before 0" as -1, "two ticks before 0" as -2. Same ink, two readings - signed and unsigned are not different numbers, they're different *questions you ask about the same number*.

The top half of the dial (bit 255 set) IS the negative half. ``slt`` vs ``lt``, ``sdiv`` vs ``div`` - later ops differ only in which half of the dial they take seriously. Your ``operator`` already produces correct signed results for free; signed ops are just reading conventions.

Comparing: find the first disagreement

Don't compare numbers - compare wheels, starting from the big end. The first wheel that disagrees settles everything; nothing to its right can overrule it.

```

a:  [0] [0] [5] [9]      b:  [0] [0] [7] [1]
                                ^
wheel 3:  0 = 0  tie      -> keep walking
wheel 2:  0 = 0  tie      -> keep walking
wheel 1:  5 < 7  DECIDED  -> a < b, wheel 0 never consulted

```

It's how you'd compare ``4700`` and ``4699`` by eye - the hundreds digit ends the argument. Write ``<`` once; ``>``, ``<=``, ``>=`` are one-line rearrangements of it.

Bitwise: the ops where wheels don't talk

``&``, ``|``, ``^``, ``~`` are the only operations with **no clicks, no borrows, no spillover** - bit k of the answer depends only on bit k of the two inputs. Each wheel is an independent 64-switch panel:

```
r.wheel[i] = a.wheel[i] & b.wheel[i]    // four native ANDs. done.
```

That's why they're the easy warm-up: the machinery of carries that makes arithmetic hard simply isn't present.

Shifting: the conveyor belt

Now imagine all 256 bits as beads on a conveyor belt. `<< n`` means "pull the belt *n* places to the left." Two things happen at once, and keeping them separate is the whole skill:

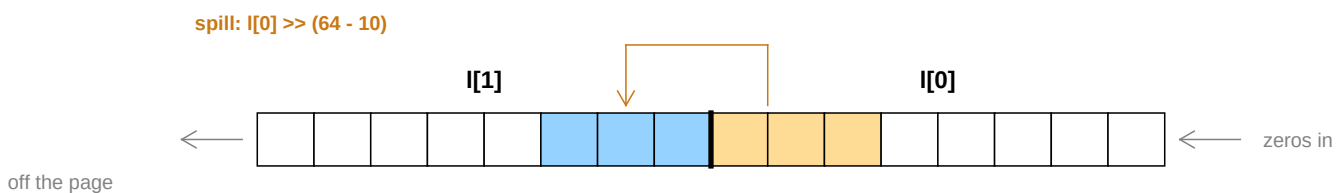
1. ****Beads that reach the left edge fall off**** - off the page, gone.
(Zeros feed in from the right to fill the gap.)
2. ****The wheels aren't welded shut**** - a bead sliding off the top of wheel 0 doesn't vanish; it lands at the bottom of wheel 1.

Every shift `n`` splits into `n = 64*(n/64) + (n%64)``:

```
n/64  = how many wheels slide over whole
n%64  = the leftover slide within a wheel
```

Example: `<< 74`` = slide one whole wheel, plus 10 bits within each wheel.
Destination wheel `i`` gets its content from TWO sources - like catching rain in a gutter that also receives drips from the roof below:

```
dest wheel i = (src wheel i-1 slid up 10 bits) -- main slide
               | (top 10 bits of wheel i-2)    -- spill through
                                           the doorway
```



The spill piece is `a.l[src-1] >> (64 - 10)`` - take the source's top 10 beads, walk them down to the bottom, and OR them into the gap the slide left open. `>>>` is the mirror: belt pulls right, beads fall off wheel 0's right edge, zeros feed into wheel 3's top.

****The trap:**** when the leftover is 0 (`<< 64``, `<< 128``), the spill formula computes `>>> 64`` - and shifting a 64-bit value by 64 is undefined behavior (the hardware quietly masks it to `>>> 0`` and leaks garbage). Guard every spill term with `if (bit_shift && ...)``. This is the classic multi-limb bug: it looks right, passes half the tests, and corrupts exact-multiple-of-64 shifts.

Addressing: which wheel, which tick

`bit(a, i)``, `byte_at``, `bit_length`` are all the same question -
"given a bit index, which wheel and which position?"

```
wheel    = i / 64      which wheel holds bit i
position = i % 64      where on that wheel

bit(a, i) = (a.l[i/64] >> (i%64)) & 1    // slide it to the end,
```

```

// mask everything else
set bit i: q.l[i/64] |= 1 << (i%64) // OR a single-bead mask

```

One deliberate quirk: `byte\_at` counts from the **big end** - byte 0 is the most significant byte - because the EVM's BYTE opcode reads the number the way a human reads `4702` (left to right). Everything else in the room counts from the small end. Keep the two conventions separate.

Multiplying: the area picture

Here's the 3blue1brown moment. Don't think "multiply digits" - think **area**. `a \* b` is the area of a rectangle with sides `a` and `b`. Cut each side into its four wheels, and the big rectangle tiles into 16 small ones:

	a3	a2	a1	a0
b3	a3b3	a2b3	a1b3	a0b3
b2	a3b2	a2b2	a1b2	a0b2
b1	a3b1	a2b1	a1b1	a0b1
b0	a3b0	a2b0	a1b0	a0b0

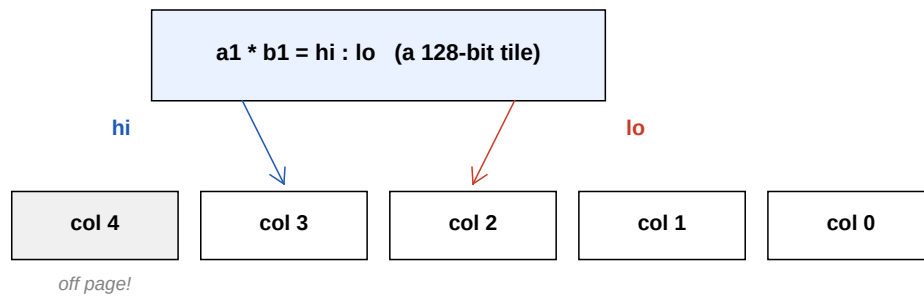
Now the key question - *where does each tile's area belong?* Wheel `ai` has weight B^i , wheel `bj` has weight B^j , so tile `(i,j)` is a rectangle B^i wide and B^j tall: its area is $a_i b_j * B^{(i+j)}$. **The place value isn't a rule to memorize - it's geometry.** Tiles on the same anti-diagonal ($i+j$ equal) share a place value and stack into the same wheel.

One more physical fact: a wheel-times-wheel product doesn't fit on a wheel. $9 \times 9 = 81$ - a digit times a digit gives a **two-digit** answer. Ours is worse: 64-bit x 64-bit = up to 128 bits = **two wheels**, a low half and a high half. `\_umul128` is the hardware giving you both wheels of that sub-answer at once:

```

ai * bj = hi : lo      (a 128-bit tile)
|      +-- lands on wheel i+j
+----- lands on wheel i+j+1 (one position up, always)

```



Walkthrough - $U(0,1,0,0) * U(0,1,0,0) = 2^{64} \times 2^{64} = 2^{128}$:

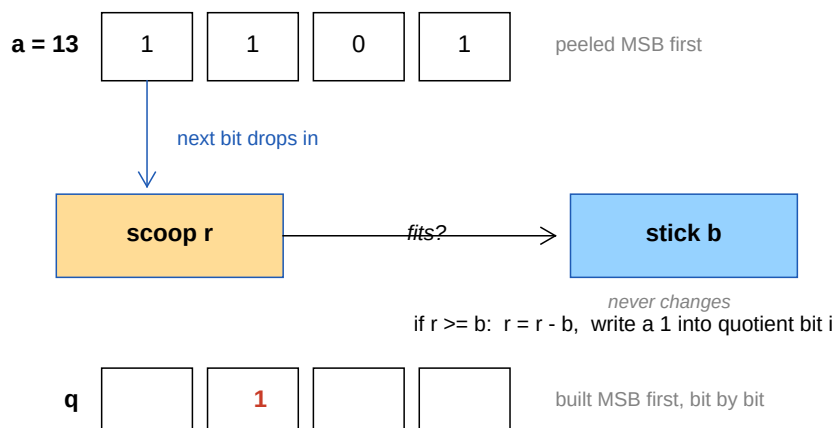
```
only nonzero tile: i=1, j=1 -> lands on wheel 2
1 * 1 -> hi=0, lo=1
result: U(0, 0, 1, 0) [checkmark]
```

Carries here are different from addition: the thing flowing right-to-left isn't a flag, it's hi - a whole wheel's worth, up to $B-2$ (since the biggest tile is $(B-1)^2 = (B-2)B + 1$ - the "9x9=81, you can never carry a 10" argument). Tiles with $i+j \geq 4$ are off the page - the rectangle's top-right corner has no wheels to land on. Skipped: that's the mod, built into the loop bounds.

Dividing: the measuring stick

Division answers a child's question: *how many times does the stick b fit along the rope a ?* In decimal long division you guess each digit - how many times does 37 go into 128? Binary removes the guesswork: **the quotient digit can only be 0 or 1.** Either the stick fits or it doesn't.

The machine: build the answer one bit at a time, most significant first. Keep a "remainder scoop" r . Each round, slide the scoop one place and drop the next bit of a into it - the "bring down the next digit" step from school. Then ask the only question: does the stick fit in the scoop? If yes, subtract it and write a 1 in the quotient; if no, write a 0 and move on.



Walkthrough - `13 / 3` (`a = 1101`, `b = 0011`):

i=3:	scoop = 0<<1 1 = 1	stick fits? no	q = 0000
i=2:	scoop = 1<<1 1 = 3	fits? yes -> scoop = 0	q = 0100
i=1:	scoop = 0<<1 0 = 0	fits? no	q = 0100
i=0:	scoop = 0<<1 1 = 1	fits? no	q = 0100

q = 4, scoop left over = 1 check: 13 = 4*3 + 1 [checkmark]

`r = (r << 1) | bit(a, i)` is "slide the scoop, drop the next bit in"
 - the shift opens a slot at the bottom, the OR drops the bit into it.
 `%` is the same machine returning the scoop instead of the quotient.

Rulebook quirks: `x / 0 = 0`, `x % 0 = 0`. The clerk doesn't panic -
 division by zero is a defined answer, not an error.

Signed ops: same dial, different reading

No new machinery - `slt`, `sgt`, `sdiv`, `smod` are the clock-face section applied. Bit 255 = sign. Signed compare: if signs differ, the negative one is smaller; if same sign, compare as usual (remembering that among negatives, bigger magnitude = more negative). Signed division: do unsigned division on the magnitudes, then paint the sign back on - quotient's sign = XOR of the signs, remainder follows the dividend's sign.

addmod / mulmod: the hidden wide desk

Trap ops. `(a + b) mod n` looks like "add, then mod" - but `a + b` can roll the odometer, and the rolled-over reading is a *different number* mod `n`:

addmod(max, max, 3):		
naive:	(max + max) wraps to $2^{256} - 2$; mod 3 = 2	WRONG
true:	$2^{257} - 2$ mod 3 = 0	RIGHT

The clerk needs a hidden wider worksheet - a 257-bit (for add) or 512-bit (for mul) scratch area that never goes on the visible desk.
 `mulmod` is why the multiplication chapter matters so much: you need the FULL 512-bit rectangle before you can fold it mod `n`.

Conversions: the direction you read

Inside the room: little-endian wheels (wheel 0 is small). On paper, on the wire, in test vectors: big-endian (most significant byte first, like `4702`). Every conversion is just walking in the right direction:

from_bytes(p):	p[0] is the BIG end -> top of wheel 3
	p[n-1] is small end -> bottom of wheel 0
to_bytes32:	reverse walk, pad zeros on the LEFT

```
from_hex:      digits arrive big-end-first too
```

Rules of the room

- The odometer has four wheels and no fifth: everything wraps mod 2^{256} .
- Carries and borrows between wheels are real; off the edge, forgiven.
- A wheel is a digit in base 2^{64} . Position IS place value.
- Wheel x wheel = two wheels. Quotient digits are only 0 or 1.
- The clerk never panics: $x/0 = 0$, and a clean stop is a valid answer.
- Signed and unsigned are two readings of one dial, not two numbers.